

CESIZen

Cahier de tests

Projet	CESIZen — Application de santé mentale
Référentiel	Concepteur Développeur d'Applications (CDA)
Bloc	Bloc 2 — Développer et tester les applications
Auteur	Tim LOPES
Version du document	1.0
Périmètre	API Laravel + Mobile Flutter + Back-office Web

1. Objectif et stratégie de test

Ce cahier de tests décrit la stratégie mise en place pour garantir la qualité du projet **CESIZen**. Il liste les scénarios à exécuter avant chaque livraison et fournit le modèle de procès-verbal de recette à compléter à l'issue de la campagne.

La démarche de test s'articule autour de **quatre niveaux complémentaires** :

- **Tests unitaires** : valident en isolation le comportement d'une fonction ou d'une classe (ex. accesseurs Eloquent, helpers, validation métier).
- **Tests fonctionnels** : valident un endpoint ou un workflow utilisateur de bout en bout (HTTP → contrôleur → service → BDD).
- **Tests d'intégration** : vérifient l'interaction réelle avec la base de données et les services externes.
- **Tests de non-régression** : exécutés à chaque évolution pour s'assurer que les fonctionnalités existantes restent opérationnelles.

Outils utilisés

Outil	Cible	Type de test
PHPUnit 11	API Laravel	Unitaires + Fonctionnels
flutter_test	Mobile + Back-office	Unitaires
Postman / Bruno	API	Manuel exploratoire
Swagger UI	API	Validation contractuelle
Plateformes physiques	Mobile (Android/iOS)	Recette manuelle

Légende des statuts

Statut	Signification
✓ OK	Test passé avec succès, comportement conforme à l'attendu
✗ KO	Test échoué, anomalie à corriger avant la livraison
■ Partiel	Test passé avec une réserve mineure (ex. UX à améliorer)
■ Non joué	Test non exécuté lors de cette campagne

2. Environnements de test

Environnement	Description	URL / Accès
Local développeur	Stack Docker complète sur la machine du développeur	<code>http://localhost:8000</code>
Émulateur Android	Application mobile Flutter exécutée via Android Studio	<code>http://10.0.2.2:8000</code> (vers API hôte)
Recette	Environnement intermédiaire avant production, données réelles	(à provisionner)
Production	Environnement final, données réelles, surveillé	(à provisionner)

Jeu de données de test

Les tests exécutent un jeu de données reproductible défini dans les seeders Laravel (database/seeders/) :

- 2 rôles : *admin* et *user*
- 3 états utilisateur : *active*, *inactive*, *banned*
- 2 utilisateurs initiaux : `admin@cesizen.fr` et `demo@cesizen.fr`
- 6 catégories d'émotions × 5 à 6 émotions chacune
- 3 exercices de respiration : 7-4-8, 5-5, 4-6
- 5 pages d'information éditoriales

Tous les tests fonctionnels utilisent RefreshDatabase et SQLite en mémoire, ce qui garantit l'isolation entre tests et l'absence d'effet de bord.

3. Tests unitaires

Les tests unitaires valident le comportement isolé des classes métier, indépendamment d'une connexion HTTP ou d'une session utilisateur. Ils sont localisés dans tests/Unit/ et exécutés via php artisan test --testsuite=Unit.

ID	Nom	Description	Étapes	Résultat attendu	Resp.	Statut
TU-001	isAdmin()	Le helper isAdmin() du modèle User retourne true si le rôle est <i>admin</i> .	1. Créer un User avec id_role = admin 2. Charger la relation role 3. Appeler isAdmin()	Retourne true isUser() retourne false	Dev	✓ OK
TU-002	full_name accessor	L'accessor full_name concatène prénom + nom.	1. Créer un User (Jean / Dupont) 2. Lire \$user->full_name	"Jean Dupont"	Dev	✓ OK
TU-003	isActive()	Le helper renvoie false pour un utilisateur dans l'état <i>inactive</i> .	1. Créer User avec id_user_state = inactive 2. Appeler isActive()	Retourne false	Dev	✓ OK
TU-004	Cast 'hashed'	Le mot de passe est automatiquement hashé via le cast Eloquent.	1. Créer User avec password = 'secret' 2. Comparer en BDD	Le hash bcrypt est stocké, jamais en clair	Dev	✓ OK
TU-005	Période 'week'	Le service EmotionJournalService calcule correctement la plage 'semaine'.	1. Appeler getReport(period='week') 2. Vérifier from/to retournés	from = aujourd'hui - 6 jours to = aujourd'hui	Dev	✓ OK

4. Tests fonctionnels — Authentification

Les tests fonctionnels exécutent un appel HTTP réel via le client de test PHPUnit / Laravel et vérifient la réponse complète (statut, structure JSON, effet en base).

ID	Nom	Description	Étapes	Résultat attendu	Resp.	Statut
TF-001	Inscription valide	Un visiteur peut créer un compte avec des données valides.	1. POST /api/auth/register avec email + password 8+ car 2. Vérifier la réponse	HTTP 201 Token retourné User en BDD	Dev	✓ OK
TF-002	Inscription confirmation KO	L'inscription échoue si la confirmation du mot de passe ne correspond pas.	1. POST /api/auth/register 2. password ≠ password_confirmation	HTTP 422 Erreur 'password' retournée	Dev	✓ OK

Nom	Description	Étapes	Résultat attendu	Resp.
Connexion valide	Un utilisateur actif peut se connecter avec ses identifiants.	1. POST /api/auth/login 2. email + password corrects	HTTP 200 Token + user retournés	Dev
Connexion mauvais mdp	La connexion échoue avec un mot de passe incorrect.	1. POST /api/auth/login 2. password incorrect	HTTP 401 Pas de token	Dev
Utilisateur inactif	Un utilisateur dans l'état 'inactive' ne peut pas se connecter.	1. Créer User inactive 2. Tenter login	HTTP 403 Message 'Compte inactif'	Dev
/auth/me	Un utilisateur authentifié peut récupérer ses informations.	1. GET /api/auth/me 2. Header Authorization: Bearer {token}	HTTP 200 JSON user complet	Dev
Logout	Le logout révoque le token courant.	1. POST /api/auth/logout 2. Réutiliser le token	Logout : HTTP 200 Réutilisation : HTTP 401	Dev
Forgot password	Le flow forgot-password renvoie toujours un 200 (anti-énumération).	1. POST /api/auth/forgot-password 2. Email existant ET inexistant	HTTP 200 dans les 2 cas	Dev

5. Tests fonctionnels — Journal d'émotions

Nom	Description	Étapes	Résultat attendu	Resp.
Création entrée	Un utilisateur connecté peut créer une entrée dans son journal.	1. POST /api/journal 2. id_emotion + entry_date	HTTP 201 Entrée créée id_user automatique	Dev
Date future refusée	L'API refuse une date d'entrée postérieure à aujourd'hui.	1. POST /api/journal 2. entry_date = today + 3	HTTP 422 Erreur de validation	Dev
Liste personnelle	Un utilisateur ne voit que ses propres entrées.	1. User A crée 1 entrée 2. User B liste 3. Vérifier que B ne voit rien	User A : 1 entrée User B : 0 entrée	Dev
Modification	L'utilisateur peut modifier sa propre entrée.	1. PUT /api/journal/{id} 2. Nouveau note	HTTP 200 Note mise à jour en BDD	Dev
Suppression	L'utilisateur peut supprimer sa propre entrée.	1. DELETE /api/journal/{id} 2. GET liste	HTTP 204 Entrée absente de la liste	Dev
Cloisonnement modif	Un user A ne peut pas modifier l'entrée d'un user B.	1. User B crée une entrée 2. User A tente PUT sur cette entrée	HTTP 403 ou 404	Dev
Rapport semaine	Le rapport agrégé renvoie la structure attendue.	1. GET /api/journal/report?period=week 2. Vérifier la structure JSON	Clés : from, to, total_entries, by_category[]	Dev
Rapport plage perso	Le rapport accepte une plage from/to personnalisée.	1. GET /api/journal/report?from=YYYY-MM-DD&to;=YYYY-MM-DD	HTTP 200 Données filtrées sur la plage	Dev

6. Tests fonctionnels — Espace administrateur

Nom	Description	Étapes	Résultat attendu	Resp.
Accès admin refusé	Un utilisateur non-admin ne peut pas accéder aux routes admin.	1. Login en tant que user 2. GET /api/admin/users	HTTP 403	Dev
Liste utilisateurs	Un admin peut lister les utilisateurs avec pagination.	1. Login admin 2. GET /api/admin/users?page=1	HTTP 200 Pagination + total	Dev

Nom	Description	Étapes	Résultat attendu	Resp.
Création user admin	Un admin peut créer un utilisateur via le back-office.	1. POST /api/admin/users 2. Données complètes	HTTP 201 User créé avec rôle attribué	Dev
Désactivation	Un admin peut désactiver un compte utilisateur.	1. PATCH /api/admin/users/{id}/deactivate 2. User tente login	Désactivation HTTP 200 Login HTTP 403	Dev
Suppression user	Un admin peut supprimer un utilisateur (soft-delete).	1. DELETE /api/admin/users/{id} 2. Vérifier deleted_at	HTTP 204 Champ deleted_at non null	Dev
Création page info	Un admin peut créer une page d'information.	1. POST /api/admin/info-pages 2. slug, title, content HTML	HTTP 201 Page accessible côté Front	Dev
Slug dupliqué	Le slug d'une page d'information est unique.	1. Créer page slug=A 2. Créer 2e page slug=A	2e tentative : HTTP 422	Dev
Page non publiée	Une page is_published=false n'apparaît pas côté public.	1. Créer page brouillon 2. GET /api/info-pages	La page n'est pas listée	Dev
Dashboard KPIs	L'endpoint dashboard retourne tous les KPIs.	1. GET /api/admin/dashboard	Clés : users, info_pages, emotions, journal, breathing	Dev
CRUD émotions	Un admin peut gérer le référentiel d'émotions.	1. Créer catégorie 2. Ajouter émotion 3. Modifier 4. Supprimer	Toutes les opérations renvoient 2xx	Dev
CRUD respiration	Un admin peut gérer les exercices de respiration.	1. POST 2. PUT 3. DELETE sur /api/admin/breathing-exercises	Tous renvoient 2xx BDD à jour	Dev

7. Recette manuelle — Front-Office mobile

Les scénarios suivants sont à exécuter **manuellement** sur l'application Flutter via un émulateur Android et/ou un appareil iOS, après que l'API soit lancée et seedée.

Nom	Description	Étapes	Résultat attendu	Resp.
Visiteur anonyme	Un visiteur peut consulter les pages d'info et les exercices sans compte.	1. Lancer l'app sans compte 2. Ouvrir 'Infos' et 'Cohérence'	Les 2 sections sont accessibles Pas de redirection /login	QA
Tentative journal	Un visiteur tentant d'accéder au journal est redirigé.	1. Tap onglet 'Journal' sans être connecté	Affichage 'Compte requis' + bouton Login	QA
Inscription mobile	Un nouvel utilisateur peut s'inscrire depuis l'app.	1. Login → Inscrivez-vous 2. Remplir formulaire 3. Valider	Redirection vers Accueil Token persistant après relance	QA
Saisie émotion	Un utilisateur peut saisir une émotion depuis le journal.	1. Onglet Journal → FAB + 2. Choisir date, émotion, note 3. Enregistrer	Snackbar de succès Entrée visible dans l'historique du jour	QA
Rapport semaine	La page Accueil affiche l'émotion dominante et le top 3.	1. Saisir 5 entrées sur la semaine 2. Ouvrir l'Accueil	Cercle pastel + 'Vous êtes...' 3 anneaux pourcentage	QA
Cohérence cardiaque	L'animation respiratoire est synchronisée avec les phases.	1. Choisir 7-4-8 2. Démarrer 3. Observer la bulle	Bulle se gonfle 7s, statique 4s, dégonfle 8s Compteur 5min décompté	QA
Modification profil	L'utilisateur peut modifier ses informations.	1. Profil → Modifier 2. Changer ville 3. Enregistrer	Snackbar de succès Donnée à jour à l'écran et après reload	QA

Nom	Description	Étapes	Résultat attendu	Resp.
Suppression compte	L'utilisateur peut supprimer son compte (RGPD).	1. Profil → Supprimer 2. Confirmer le dialog	Déconnexion + redirection Accueil visiteur Connexion impossible avec ces ids	QA
Mot de passe oublié	Le flow forgot-password n'expose pas l'existence du compte.	1. Login → Mot de passe oublié 2. Email inconnu puis connu	Même message dans les 2 cas	QA

8. Tests de non-régression

Les scénarios de non-régression sont à rejouer **avant chaque livraison** pour valider qu'aucune fonctionnalité n'a été dégradée par les évolutions récentes.

ID	Scénario	Périmètre vérifié	Résultat attendu	Statut
NR-001	Les 4 tests Auth automatisés passent	Authentification API	AuthTest : 5/5 verts	✓ OK
NR-002	Les 5 tests Journal automatisés passent	Journal d'émotions	EmotionJournalTest : 5/5 verts	✓ OK
NR-003	Migrations + seeders s'exécutent sans erreur	Schéma BDD + données initiales	php artisan migrate --seed renvoie OK	✓ OK
NR-004	Login admin → accès au dashboard	Back-office Web	Login OK + Dashboard chargé sans erreur console	■ Non joué
NR-005	Login user → mobile fonctionne	Front-Office mobile	Token persistant + page Accueil chargée	■ Non joué
NR-006	Swagger UI accessible et fonctionnel	Documentation API	Page chargée Authentification testable	■ Non joué
NR-007	3 exercices de respiration jouent leur cycle	Module bonus	7-4-8, 5-5, 4-6 fonctionnent sur 1 cycle complet	■ Non joué
NR-008	Suppression user → soft delete	Conformité RGPD	deleted_at non null connexion impossible	■ Non joué

9. Procès-verbal de recette

Modèle de procès-verbal à compléter et signer à l'issue de chaque campagne de recette.

Champ	Valeur
Date de la recette	
Version testée	
Environnement	☐ Local ☐ Recette ☐ Production
Testeur(s)	
Nombre de tests joués	
Nombre de tests OK	
Nombre de tests KO	
Nombre de tests partiels	
Anomalies bloquantes ouvertes	
Décision	☐ Accepté ☐ Accepté avec réserves ☐ Refusé
Commentaire général	

Signatures

Rôle	Nom	Date	Signature
Développeur			
Recetteur			
Responsable produit			

10. Annexes

A. Commandes utiles

Lancer la suite complète :

```
docker exec -it cesizen_app php artisan test
```

Lancer uniquement les tests unitaires :

```
docker exec -it cesizen_app php artisan test --testsuite=Unit
```

Lancer un test spécifique :

```
docker exec -it cesizen_app php artisan test --filter=test_login_with_valid_credentials
```

B. Couverture de code

Pour générer un rapport de couverture HTML (nécessite Xdebug ou pcov) :

```
docker exec -it cesizen_app php artisan test --coverage-html coverage/
```

Le rapport est consultable dans `storage/app/coverage/index.html`.